



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2223 - TEORÍA DE AUTÓMATAS Y LENGUAJES FORMALES
PROFESOR: CRISTIAN RIVEROS
AYUDANTE: SANTIAGO MARCANO

Ayudantía 4

Repaso II

Pregunta 1

Dado $\Sigma = \{a, b\}$, construya una expresión regular para el lenguaje de todas las palabras w que contienen exactamente una posición i tal que la letra en la posición i es igual a la letra en la posición $i + 1$.

Es decir, las letras de w deben alternar entre a y b , excepto exactamente una vez, donde aparecen dos letras iguales consecutivas.

Solución

Si existe exactamente una posición donde aparecen dos letras iguales consecutivas, existen dos casos: la única repetición es aa o la única repetición es bb .

Consideremos primero el caso donde la única repetición es aa . Antes de esta repetición las letras deben alternar antes de aa pero asegurando que este bloque alterando debe terminar con una b ser una b . El bloque de alternar puede empezar con a o con b . Las palabras de esta forma se pueden describir mediante $(\varepsilon + b)(ab)^*$.

De manera similar, después de aa las letras deben seguir alternando y, si existe alguna letra después de aa , la primera debe ser una b . También el bloque puede terminar con una a o una b . Esta parte se puede describir mediante $(ba)^*(\varepsilon + b)$.

Por lo tanto, las palabras cuya única repetición es aa son descritas por

$$(\varepsilon + b)(ab)^*aa(ba)^*(\varepsilon + b).$$

El caso donde la única repetición es bb es simétrico por lo que obtenemos

$$(\varepsilon + a)(ba)^*bb(ab)^*(\varepsilon + a).$$

Juntando ambos casos, una expresión regular para el lenguaje es

$$(\varepsilon + b)(ab)^*aa(ba)^*(\varepsilon + b) + (\varepsilon + a)(ba)^*bb(ab)^*(\varepsilon + a).$$

Pregunta 2

Sea L un lenguaje regular sobre un alfabeto Σ y sea $\# \notin \Sigma$ un símbolo nuevo. Definimos el lenguaje

$$L^{\text{mr}} = \{uv\#a \mid u, v \in \Sigma^*, a \in \Sigma \text{ y } uav \in L\}.$$

Demuestre que L^{mr} es regular sobre el alfabeto $\Sigma \cup \{\#\}$.

Solución

Como L es regular, existe un DFA $A = (Q, \Sigma, \delta, q_0, F)$ tal que $L(A) = L$.

La idea será construir un ε -NFA que, mientras lee la parte uv , adivine en qué posición se encontraba originalmente la letra a . Al realizar esta elección, el autómata simulará la lectura de a mediante una transición ε y recordará cuál fue la letra escogida. Finalmente, después de leer $\#$, verificará que la última letra de la entrada sea precisamente la letra recordada.

Construimos el ε -NFA $N = (Q', \Sigma \cup \{\#\}, \Delta, I, F')$.

Para cada $a \in \Sigma$, utilizaremos estados de la forma (q, a) , donde $q \in Q$. El segundo componente recuerda la letra que fue eliminada de su posición original.

Además, para cada $a \in \Sigma$ agregamos un nuevo estado h_a y un nuevo estado final q_f . De esta forma,

$$Q' = Q \cup (Q \times \Sigma) \cup \{h_a \mid a \in \Sigma\} \cup \{q_f\}.$$

El único estado inicial es $I = \{q_0\}$ y el único estado final es $F' = \{q_f\}$.

Ahora definimos las transiciones de N , recordando que la notación $(p, a, q) \in \Delta$ significando estando en el estado p , leyendo la letra a nos movemos al estado q :

1. Mientras todavía no se ha escogido la letra que será movida al final, el autómata simplemente simula a A . Para todo $q \in Q$ y $b \in \Sigma$, agregamos

$$(q, b, \delta(q, b)) \in \Delta.$$

2. En cualquier momento, el autómata puede adivinar que la letra eliminada de su posición original era alguna $a \in \Sigma$. Para ello, simula la lectura de a en A sin consumir ninguna letra de la entrada y pasa a recordar a . Para todo $q \in Q$ y $a \in \Sigma$, agregamos

$$(q, \varepsilon, (\delta(q, a), a)) \in \Delta.$$

3. Una vez escogida la letra a , el autómata continúa simulando a A mientras mantiene a en memoria. Para todo $q \in Q$ y $a, b \in \Sigma$, agregamos

$$((q, a), b, (\delta(q, b), a)) \in \Delta.$$

4. Cuando aparece el símbolo $\#$, la simulación de A debe encontrarse en un estado final. Por lo tanto, para todo $f \in F$ y $a \in \Sigma$, agregamos

$$((f, a), \#, h_a) \in \Delta.$$

5. Finalmente, desde h_a solo podemos terminar leyendo precisamente la letra a que fue recordada:

$$(h_a, a, q_f) \in \Delta$$

para todo $a \in \Sigma$.

Ahora debemos demostrar que

$$L(N) = L^{\text{mr}}$$

mediante doble contención.

$$L^{\text{mr}} \subseteq L(N)$$

Sea $w \in L^{\text{mr}}$. Por definición de L^{mr} , existen $u, v \in \Sigma^*$ y $a \in \Sigma$ tales que

$$w = uv\#a \quad \text{y} \quad uav \in L.$$

Escribamos $u = x_1x_2 \cdots x_m$ y $v = y_1y_2 \cdots y_n$.

Como $uav \in L = L(A)$, existe una ejecución de aceptación de A sobre uav de la forma

$$\begin{aligned} \rho : \quad p_0 &\xrightarrow{x_1} p_1 \xrightarrow{x_2} \cdots \xrightarrow{x_m} p_m \xrightarrow{a} r_0 \\ &\xrightarrow{y_1} r_1 \xrightarrow{y_2} \cdots \xrightarrow{y_n} r_n, \end{aligned}$$

donde $p_0 = q_0$ y $r_n \in F$.

En particular, las transiciones de esta ejecución satisfacen:

- $\delta(p_{i-1}, x_i) = p_i$ para todo $i \in \{1, \dots, m\}$,
- $\delta(p_m, a) = r_0$,
- $\delta(r_{j-1}, y_j) = r_j$ para todo $j \in \{1, \dots, n\}$.

Consideremos ahora en N la ejecución

$$\begin{aligned} \rho' : \quad p_0 &\xrightarrow{x_1} p_1 \xrightarrow{x_2} \cdots \xrightarrow{x_m} p_m \xrightarrow{\varepsilon} (r_0, a) \\ &\xrightarrow{y_1} (r_1, a) \xrightarrow{y_2} \cdots \xrightarrow{y_n} (r_n, a) \xrightarrow{\#} h_a \xrightarrow{a} q_f. \end{aligned}$$

Veamos que todas estas transiciones pertenecen a Δ :

- Para cada $i \in \{1, \dots, m\}$, como $\delta(p_{i-1}, x_i) = p_i$, por construcción de N existe la transición

$$p_{i-1} \xrightarrow{x_i} p_i.$$

- Como $\delta(p_m, a) = r_0$, por construcción existe

$$p_m \xrightarrow{\varepsilon} (r_0, a).$$

- Para cada $j \in \{1, \dots, n\}$, como $\delta(r_{j-1}, y_j) = r_j$, por construcción existe

$$(r_{j-1}, a) \xrightarrow{y_j} (r_j, a).$$

- Como $r_n \in F$, existe la transición

$$(r_n, a) \xrightarrow{\#} h_a.$$

- Finalmente, por construcción existe

$$h_a \xrightarrow{a} q_f.$$

La ejecución ρ' comienza en q_0 , consume exactamente la palabra

$$x_1 \cdots x_m y_1 \cdots y_n \# a = uv\#a = w,$$

y termina en $q_f \in F'$.

Por lo tanto, ρ' es una ejecución de aceptación de N sobre w , y concluimos que $w \in L(N)$.

$$L(N) \subseteq L^{\text{mr}}$$

Sea $w \in L(N)$. Entonces existe una ejecución de aceptación ρ de N sobre w .

Como el único estado final de N es q_f , la ejecución debe terminar en q_f . Además, por construcción de Δ , las únicas transiciones que llegan a q_f son de la forma

$$h_a \xrightarrow{a} q_f$$

para algún $a \in \Sigma$.

Por lo tanto, la última letra de w es alguna letra $a \in \Sigma$, y justo antes de leerla la ejecución se encuentra en h_a .

A su vez, la única forma de llegar a h_a es mediante una transición

$$(f, a) \xrightarrow{\#} h_a$$

para algún $f \in F$. Luego, antes de la última letra a , la ejecución debe haber leído el símbolo $\#$ desde un estado de la forma (f, a) con $f \in F$.

Finalmente, la única forma de entrar desde un estado de Q a un estado de $Q \times \Sigma$ que recuerda la letra a es mediante una transición

$$p \xrightarrow{\varepsilon} (\delta(p, a), a)$$

para algún $p \in Q$.

Una vez que la ejecución entra a un estado de $Q \times \Sigma$, no puede volver a un estado de Q , y el segundo componente a se mantiene fijo hasta leer el símbolo $\#$.

Por lo tanto, existen palabras $u = x_1 \cdots x_m$ y $v = y_1 \cdots y_n$ tales que la ejecución de aceptación tiene necesariamente la forma

$$\begin{aligned} \rho : \quad p_0 &\xrightarrow{x_1} p_1 \xrightarrow{x_2} \cdots \xrightarrow{x_m} p_m \xrightarrow{\varepsilon} (r_0, a) \\ &\xrightarrow{y_1} (r_1, a) \xrightarrow{y_2} \cdots \xrightarrow{y_n} (r_n, a) \xrightarrow{\#} h_a \xrightarrow{a} q_f, \end{aligned}$$

donde $p_0 = q_0$ y $r_n \in F$.

Por la definición de las transiciones de N , para cada $i \in \{1, \dots, m\}$ se cumple

$$\delta(p_{i-1}, x_i) = p_i.$$

Además, como

$$p_m \xrightarrow{\varepsilon} (r_0, a)$$

es una transición de N , por construcción necesariamente

$$\delta(p_m, a) = r_0.$$

Finalmente, para cada $j \in \{1, \dots, n\}$, la transición

$$(r_{j-1}, a) \xrightarrow{y_j} (r_j, a)$$

existe solamente si

$$\delta(r_{j-1}, y_j) = r_j.$$

Por lo tanto, en el DFA A existe la ejecución

$$\begin{aligned} p_0 &\xrightarrow{x_1} p_1 \xrightarrow{x_2} \cdots \xrightarrow{x_m} p_m \xrightarrow{a} r_0 \\ &\xrightarrow{y_1} r_1 \xrightarrow{y_2} \cdots \xrightarrow{y_n} r_n. \end{aligned}$$

Esta ejecución comienza en $p_0 = q_0$ y termina en $r_n \in F$, por lo que es una ejecución de aceptación de A sobre

$$x_1 \cdots x_m a y_1 \cdots y_n = uav.$$

Luego, $uav \in L(A) = L$. concluyendo que $w \in L^{\text{mr}}$.

Finalmente, $L(N) = L^{\text{mr}}$. Como N es un ε -NFA, concluimos que L^{mr} es regular.

Pregunta 3

Considere el siguiente problema:

Problema: DIFFERENCE-NONEMPTY

Input: Dos NFAs A y B sobre un mismo alfabeto Σ .

Output: TRUE si, y solo si, existe una palabra $w \in \Sigma^*$ tal que

$$w \in L(A) \quad \text{y} \quad w \notin L(B).$$

Escriba un algoritmo para el problema DIFFERENCE-NONEMPTY. Explique la correctitud de su algoritmo.

Solución

Queremos determinar si existe una palabra que pertenezca a $L(A)$ pero no a $L(B)$. Esto es equivalente a determinar si

$$L(A) \cap L(B)^C \neq \emptyset.$$

El problema es que A y B son NFAs, y no podemos obtener el complemento de un NFA simplemente intercambiando sus estados finales y no finales. Por lo tanto, primero utilizaremos determinización.

El algoritmo es el siguiente:

1. Determinizar A y B , obteniendo DFAs A^{det} y B^{det} tales que $L(A^{\text{det}}) = L(A)$ y $L(B^{\text{det}}) = L(B)$.
2. Construir el complemento de B^{det} , que denotaremos por $\overline{B^{\text{det}}}$, intercambiando sus estados finales y no finales.
3. Construir el producto

$$C = A^{\text{det}} \times \overline{B^{\text{det}}}$$

utilizando como estados finales aquellos pares donde ambos componentes son finales. Noten que para hacer este paso es importante que A también este determinizado ya que el algoritmo que se ve en clase para hacer el producto cruz solo trabaja sobre DFA

4. Ejecutar el algoritmo de EMPTINESS sobre C .

Si $L(C) \neq \emptyset$, retornar TRUE. En caso contrario, retornar FALSE.

Correctitud

Por correctitud de la determinización, $L(A^{\text{det}}) = L(A)$ y $L(B^{\text{det}}) = L(B)$.

Como B^{det} es un DFA, al intercambiar sus estados finales y no finales obtenemos

$$L(\overline{B^{\text{det}}}) = \Sigma^* \setminus L(B).$$

Por construcción del producto para intersección,

$$\begin{aligned} L(C) &= L(A^{\text{det}}) \cap L(\overline{B^{\text{det}}}) \\ &= L(A) \cap (\Sigma^* \setminus L(B)) \\ &= L(A) \setminus L(B). \end{aligned}$$

Por lo tanto,

$$L(C) \neq \emptyset$$

si, y solo si, existe una palabra w tal que $w \in L(A)$ y $w \notin L(B)$.

Finalmente, el algoritmo de EMPTINESS retorna que $L(C)$ es no vacío si, y solo si, existe una ejecución desde el estado inicial de C hasta algún estado final.

Así, nuestro algoritmo retorna TRUE si, y solo si, existe una palabra $w \in L(A) \setminus L(B)$, que es exactamente lo requerido por DIFFERENCE-NONEMPTY.